

QUESTIONS

- Q1

Debug Nested Exception Handling

10 marks
- Q2

Debug Exception Propagation

10 marks
- Q3

Debug Multiple Runtime Exceptions

10 marks
- Q4

Debug Finally Execution

10 marks
- Q5

Debug the NullPointerException

10 marks

5/5 answered

Q1 Debug Nested Exception Handling

10 marks

```
import java.util.Scanner;

public class Debug1 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        int arr[] = {20,40,60,80};

        int index = sc.nextInt();
        int divisor = sc.nextInt();

        try {
            int value = arr[index];
            try {
                int result = value/divisor;
                System.out.println(result);
            }
            catch(ArrayIndexOutOfBoundsException e) {
                System.out.println("Invalid Index");
            }
        }
        catch(ArithmeticException e) {
            System.out.println("Division by Zero");
        }
    }
}
```

Debugging Task

Identify why some exceptions are never caught.

Modify the exception handling so all runtime exceptions are handled correctly.

TEST CASE 1

Input 2 5

Expected Output: 12

TEST CASE 2

Input 5 2

Expected Output : Invalid Index

TEST CASE 3

Input 1 0

Expected Output : Division by Zero

 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Debug1 {
3     public static void main(String args[]) {
4         Scanner sc = new Scanner(System.in);
5         int arr[] = {20, 40, 60, 80};
6         int index = sc.nextInt();
7         int divisor = sc.nextInt();
8         try {
9             int value = arr[index];
10            try {
11                int result = value / divisor;
12                System.out.println(result);
13            }
14            catch (ArithmeticException e) {
15                System.out.println("Division by Zero");
16            }
17        }
18        catch (ArrayIndexOutOfBoundsException e) {
19            System.out.println("Invalid Index");
20        }
21    }
22 }
```

Test Input

1 2

10 2

5 0

Output

20

Test Cases

- Test Case 1
- Test Case 2
- Test Case 3

10.00 / 10 marks

 Submitted

Q1

Debug Nested Exception Handling
10 marks

Q2

Debug Exception Propagation
10 marks

Q3

Debug Multiple Runtime Exceptions
10 marks

Q4

Debug Finally Execution
10 marks

Q5

Debug the NullPointerException
10 marks

5/5 answered

```
import java.util.Scanner;

class Calculator{

    static int divide(int a,int b){
        return a/b;
    }

    static void display(int x){
        System.out.println("Answer = "+x);
    }
}

public class Debug2{

    public static void main(String args[]){

        Scanner sc=new Scanner(System.in);

        int a=sc.nextInt();
        int b=sc.nextInt();

        Calculator.display(Calculator.divide(a,b));

        System.out.println("Execution Completed");
    }
}

Debugging Task
Prevent abnormal termination.
Ensure "Execution Completed" is always printed.
TEST CASE 1
INPUT: 20 5
OUTPUT : Answer = 4
Execution Completed
TEST CASE 2
INPUT: 50 0
OUTPUT: Cannot Divide by Zero
Execution Completed
TEST CASE 3
INPUT: 36 6
OUTPUT: Answer = 6
Execution Completed
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2 public class Debug2
3 {
4     public static void main(String[] args)
5     {
6         Scanner sc = new Scanner(System.in);
7         try
8         {
9             int a = sc.nextInt();
10            int b = sc.nextInt();
11            Calculator.display(Calculator.divide(a, b));
12        }
13        catch (ArithmeticException e)
14        {
15            System.out.println("Cannot Divide by Zero");
16        }
17        catch (Exception e)
18        {
19            System.out.println("Invalid Input");
20        }
21        finally
22        {
23            System.out.println("Execution Completed");
24            sc.close();
25        }
26    }
27 }
28
29 class Calculator
30 {
31     static int divide(int a, int b)
32     {
33         return a / b;
34     }
35     static void display(int x)
36     {
37         System.out.println("Answer = " + x);
38     }
39 }
```

Test Input

20 5

Output

```
Answer = 4
Execution Completed
```

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

QUESTIONS

Q1
Debug Nested Exception Handling
10 marks

Q2
Debug Exception Propagation
10 marks

Q3
Debug Multiple Runtime Exceptions
10 marks

Q4
Debug Finally Execution
10 marks

Q5
Debug the NullPointerException
10 marks

5/5 answered

Q3 Debug Multiple Runtime Exceptions

10 marks

```
import java.util.Scanner;

public class Debug3 {

    public static void main(String args[]) {

        Scanner sc=new Scanner(System.in);

        int arr[]={12,24,36,48};

        String number=sc.next();

        int index=Integer.parseInt(number);

        System.out.println(arr[index]);

    }

}
```

Debugging Task

The program may generate two different runtime exceptions.
Modify the program so that each exception is handled separately.

Test Case 1

INPUT: 2

OUTPUT: 36

Test Case 2

INPUT: ABC

OUTPUT: Invalid Number

Test Case 3

INPUT: 5

OUTPUT: Invalid Array Index

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;
2
3 public class Debug3 {
4
5     public static void main(String args[]) {
6
7         Scanner sc = new Scanner(System.in);
8
9         int arr[] = {12, 24, 36, 48};
10
11         String number = sc.next();
12
13         try {
14             int index = Integer.parseInt(number);
15             System.out.println(arr[index]);
16         } catch (NumberFormatException e) {
17             System.out.println("Invalid Number");
18         } catch (ArrayIndexOutOfBoundsException e) {
19             System.out.println("Invalid Array Index");
20         }
21     }
```

Test Input

2
ABC
5

Output

36

Test Cases

Test Case 1



Test Case 2



Test Case 3



10.00 / 10 marks

QUESTIONS

Q1
Debug Nested Exception Handling
10 marks

Q2
Debug Exception Propagation
10 marks

Q3
Debug Multiple Runtime Exceptions
10 marks

Q4
Debug Finally Execution
10 marks

Q5
Debug the NullPointerException
10 marks

5/5 answered

Q4 Debug Finally Execution

10 marks

```
public class Debug4 {  
  
    static int calculate(int a,int b){  
  
        try{  
            return a/b;  
        }  
        finally{  
            System.out.println("Resources Released");  
        }  
    }  
  
    public static void main(String args[]){  
  
        System.out.println(calculate(20,0));  
    }  
}
```

Debugging Task

Prevent program termination.

Ensure the finally block executes while handling the exception correctly.

Test Case 1

INPUT: 20 5

OUTPUT: Resources Released

4

Test Case 2

20 0

Resources Released

Division by Zero

Test Case 3

81 9

Resources Released

9

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 public class Debug4  
2 {  
3     static int calculate(int a, int b) throws ArithmeticException  
4     {  
5         try {  
6             return a / b;  
7         }  
8         finally  
9         {  
10            System.out.println("Resources Released");  
11        }  
12    }  
13    public static void main(String args[])  
14    {  
15        java.util.Scanner sc = new java.util.Scanner(System.in);  
16        if (sc.hasNextInt())  
17        {  
18            int a = sc.nextInt();  
19            int b = sc.nextInt();  
20            try  
21            {  
22                int result = calculate(a, b);  
23                System.out.println(result);  
24            }  
25            catch (ArithmeticException e)  
26            {  
27                System.out.println("Division by Zero");  
28            }  
29        }  
30        sc.close();  
31    }  
32 }
```

Test Input

5 0

Output

Resources Released
Division by Zero

Test Cases

Test Case 1

Test Case 2

Test Case 3

10.00 / 10 marks

Submitted

DEBUGGING AND OPTIMIZATION

[← Back](#)

QUESTIONS

Q1 ✓
Debug Nested Exception Handling
10 marks

Q2 ✓
Debug Exception Propagation
10 marks

Q3 ✓
Debug Multiple Runtime Exceptions
10 marks

Q4 ✓
Debug Finally Execution
10 marks

Q5 ✓
Debug the NullPointerException
10 marks

5/5 answered

Q5 Debug the NullPointerException

10 marks

```
public class Debug4 {  
    public static void main(String[] args) {  
        String name = null;  
        System.out.println(name.length());  
    }  
}
```

Expected Task

Prevent the program from crashing.

Test Case 1

INPUT: Java

OUTPUT: Length = 4

Test Case 2

INPUT: null

OUTPUT: String is Null

Test Case 3

INPUT: Programming

OUTPUT: Length = 11

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 import java.util.Scanner;  
2 public class Debug4  
3 {  
4     public static void main(String[] args)  
5     {  
6         Scanner sc = new Scanner(System.in);  
7         if (!sc.hasNext())  
8         {  
9             System.out.println("String is Null");  
10            return;  
11        }  
12        String name = sc.next();  
13        if (name.equals("null"))  
14        {  
15            System.out.println("String is Null");  
16        }  
17        else  
18        {  
19            System.out.println("Length = " + name.length());  
20        }  
21        sc.close();  
22    }  
23 }
```

✓ Submitted

Test Input

Java

Output

Length = 4

Test Cases

Test Case 1

✓

Test Case 2

✓

Test Case 3

✓

10.00 / 10 marks